Lab 5: Oscillations and Andronov-Hopf Bifurcations in a Biological Model

Stream 1: Cell Signalling

September 4, 2026

1 Goal

To investigate how a biological system can transition from a stable steady state to a state of sustained oscillations. This will be achieved by analysing a **supercritical Hopf bifurcation** as a key parameter is varied. While this lab uses a classic neuron model as its primary example, the principles of excitability and oscillation you will explore are fundamental across many physiological systems, including **calcium dynamics**, **neuroendocrine cell signalling**, and the general **regulation of cell function**. You will use Python for direct numerical simulation and, as an advanced exercise, PyDSTool for automated bifurcation analysis to visualise this phenomenon.

2 Reading material

Keener, J., & Sneyd, J. (2025). *Mathematical Physiology* (3rd ed.). Springer.

- **Chapter 5: Excitability** (primary reading for the neuronal context).
- **Chapter 7: Calcium Dynamics**.
- **Chapter 9: Neuroendocrine Cells**.

3 Background: a simplified excitable cell model

This week, we use a two-variable conductance-based model. It is a simplified representation of neuronal dynamics, similar in structure to the classic Morris-Lecar model, but using sodium and potassium currents. It describes the dynamics of the membrane potential (V) and a slower, voltage-dependent potassium channel gating variable (n). The system of ODEs is given by:

$$C_m \frac{dV}{dt} = -I_{\text{ion}}(V, n) + I_{app} \tag{1}$$

$$\frac{dn}{dt} = \phi(n_\infty(V) - n) \tag{2}$$

where the total ionic current is composed of sodium, potassium, and leak components:

$$I_{\text{ion}}(V, n) = g_{Na} m_\infty(V)(V - E_{Na}) + g_K n(V - E_K) + g_l(V - E_l) \tag{3}$$

The voltage-dependent gating functions for sodium activation (m_∞) and potassium activation (n_∞) are sigmoidal:

$$m_\infty(V) = \frac{1}{1 + \exp\left(\frac{(v_1 - V)}{k_1}\right)}$$

$$n_\infty(V) = \frac{1}{1 + \exp\left(\frac{(v_3 - V)}{k_3}\right)}$$

Use the following physiological parameters:

- $C_m = 1.0 \quad (\mu\text{F/cm}^2)$
- $g_{Na} = 20 \quad (\text{mS/cm}^2), \quad g_K = 10 \quad (\text{mS/cm}^2), \quad g_l = 8 \quad (\text{mS/cm}^2)$
- $E_{Na} = 60 \quad (\text{mV}), \quad E_K = -90 \quad (\text{mV}), \quad E_l = -80 \quad (\text{mV})$
- $v_1 = -20$ mV, $k_1 = 15$ mV
- $v_3 = -45$ mV, $k_3 = 5$ mV
- $\phi = 2.0$: timescale factor for the potassium current

4 Exercise: a complete bifurcation analysis in Python

Your task is to write a Python script that performs a full analysis of the model's dynamics as the parameter I_{app} is varied. This involves several steps that build upon each other.

Task 1: Stability analysis via the Jacobian

The first step is to determine the stability of the system's steady state at any given I_{app}. A steady state (V_{ss}, n_{ss}) is a point where $\frac{dV}{dt} = 0$ and $\frac{dn}{dt} = 0$.

1. **Find the steady state:** For a given value of I_{app}, the steady state (equilibrium) is a root of the ODE system. Use a numerical root-finder (e.g., SciPy's `fsolve`) to find the precise values of (V_{ss}, n_{ss}).
2. **Implement the Jacobian:** The stability of the steady state is determined by the eigenvalues of the Jacobian matrix, J, evaluated at that point.

   $$J = \begin{pmatrix} \frac{\partial F}{\partial V} & \frac{\partial F}{\partial n} \\ \frac{\partial G}{\partial V} & \frac{\partial G}{\partial n} \end{pmatrix}_{(V_{ss}, n_{ss})}$$

   Analytically derive the four partial derivatives of the system and create a Python function that computes this matrix.
3. **Calculate eigenvalues:** Use a numerical library (e.g., NumPy's `linalg.eigvals`) to calculate the eigenvalues, λ, of the Jacobian at the steady state. The stability is determined by the sign of the largest real part of the eigenvalues.

Task 2: Constructing the 3D bifurcation diagram

Now, create a loop that iterates through I_{app} from 0 to 400 (e.g., in steps of 5.0) to build the full diagram. Use numerical continuation by feeding the steady state found for one parameter value as the initial guess for the next.

1. **Track the steady state:** In your loop, find the steady state (equilibrium point) and calculate its eigenvalues for each I_{app}. Store these points.
2. **Identify and plot stable vs. unstable equilibria:** In a 3D plot with axes (I_{app}, V, n), plot the steady-state points. Use different markers or colors to distinguish between stable ($\max(\text{Re}(\lambda)) < 0$) and unstable ($\max(\text{Re}(\lambda)) > 0$) points.
3. **Simulate and plot limit cycles:** If a steady state is unstable, it implies a stable limit cycle exists. From a slightly perturbed initial condition near the unstable point, run a long simulation and plot the final, settled trajectory (the limit cycle) in the 3D diagram at the corresponding I_{app}. Consider making the cycles semi-transparent using an `alpha` value.
4. **Detect and label Andronov-Hopf points:** An Andronov-Hopf Bifurcation (HB) occurs when the real part of the eigenvalues crosses zero. Detect these crossings in your loop.
   - At these points, label them "HB" in your 3D plot.
   - The imaginary part of the critical eigenvalue, ω, gives the onset frequency of the oscillation. Calculate this frequency in Hertz ($f = (\omega \cdot 1000)/2\pi$) and add it to the label.

Task 3: Visualising the dynamics

Finally, create detailed 2D plots for a specific value of I_{app} in the oscillatory regime (e.g., $I_{app} = 150$) to connect the bifurcation diagram to the system's actual behavior.

1. **Phase portrait with vector field:**
    - Create a phase portrait showing the stable limit cycle and the unstable steady state.
    - Add the system's nullclines.
    - Add a vector field to show the flow. For a "fast-slow" system like this one, a standard `quiver` plot can be misleading. Use a `streamplot` instead, as it is specifically designed to visualize the direction of flow without being distorted by the speed of the dynamics.
2. **Time trace:**
    - Create a time series plot showing $V(t)$ for a short time window (e.g., 10 ms) to clearly see the oscillations.

Part B : the automated approach with PyDSTool

Now that you understand the mechanics, use PyDSTool's built-in tools to see how this entire analysis can be performed automatically.

1. **Set up the ode system** Create an ODE that implements the model equations and parameters. Set $\phi = 2$.
2. **Run continuation:** Find a steady state for an initial value of I_{app} and then run the continuation in the `forward` and `backward` direction.
3. **Trace the limit cycles:** PyDSTool will automatically detect and label the Andronov-Hopf bifurcation (HB) points. Select one of these points as the starting point for the limit-point curve (LP-C) to find the period solutions.

Discussion

1. Explain the role of the Jacobian matrix. How do its eigenvalues tell you about the stability and behavior of a steady state?
2. Compare the oscillation frequency you calculated from the eigenvalue at the first Andronov-Hopf point with the frequency you 'measured' from the time trace at $I_{app} = 150$. Are they the same? Why or why not?
3. Compare the final diagrams from your Python script and PyDSTool. What information did AUTO (PyDSTool uses AUTO) provide automatically that was difficult to obtain or calculate in your Python script (e.g., the unstable branch, precise bifurcation locations)?

Concluding remarks: the right tool for the job

Congratulations on completing this in-depth analysis! By building this entire workflow from scratch in Python, you have gained a fundamental understanding of how bifurcation analysis is performed.

The important take-home message, however, is that all of the Python tasks you meticulously implemented are already built into specialised software like **PyDSTool**. This exercise highlights the immense power and efficiency of using dedicated scientific software for complex analysis. While building methods from scratch is invaluable for learning, knowing when and how to use powerful, specialized tools is a key skill for any scientist.